

To What Extent Do Random Forest and LSTM Justify Their Additional Complexity over Rolling Historical Volatility and GARCH(1,1) When Forecasting the Next-Day Update of a Volatility Proxy Constructed from Hyperliquid BTC Perpetual-Futures Returns?

Abstract

This project compares rolling volatility, GARCH(1,1), lagged linear regression, Random Forest and Long Short-Term Memory (LSTM) for forecasting the next update of a daily Bitcoin volatility proxy. The data comprise completed Hyperliquid BTC perpetual-futures candles. The main target is the next day's updated 30-day standard deviation of daily log returns. Adjacent targets share 29 returns, so this is an update forecast rather than a forecast of a completely new 30-day period. The main test uses a fixed chronological split, followed by checks with a 14-day target, different time periods, volatility groups, expanding windows, a block bootstrap and a fixed-cutoff data refresh. The models are compared using RMSE, MAE and QLIKE, together with ease of explanation and practical cost.

GARCH(1,1) has the lowest RMSE (0.00098301) and QLIKE (0.00445845). Its RMSE is 29.569% below rolling, and it ranks first in every additional check. Neither machine-learning model beats rolling overall. Under this design, Random Forest and LSTM do not improve the forecast enough to justify their extra complexity. This conclusion applies only to the data, target, inputs and model settings used here.

1. Introduction

Volatility describes the scale of variation in returns rather than the direction of price movement. A price-direction forecast asks whether Bitcoin will rise or fall; a volatility forecast asks how uncertain or variable the next period may be. Traders and risk managers can use such an estimate when setting position sizes, risk limits or collateral buffers even when they do not know the next return's sign. Cryptocurrency is useful for this investigation because sharp movements and persistent market stress make changing uncertainty visible, while public exchange data make an independent comparison reproducible.

The research question is:

To what extent do Random Forest and LSTM justify their additional complexity over rolling historical volatility and GARCH(1,1) when forecasting the next-day update of a volatility proxy constructed from Hyperliquid BTC perpetual-futures returns?

The aim is to determine whether the additional complexity of Random Forest and LSTM over rolling historical volatility and GARCH(1,1) is justified when forecasting this next-day update. The objectives are to:

1. construct a reproducible 14- and 30-day rolling standard-deviation proxy from completed Hyperliquid BTC perpetual-futures candles;
2. implement rolling historical volatility, GARCH(1,1), lagged linear regression, scikit-learn Random Forest and LSTM without test leakage;
3. compare forecast accuracy using RMSE, MAE and QLIKE;
4. test robustness across target windows, chronological periods, volatility regimes, resampling, rolling-origin refits and a fixed-cutoff refresh;
5. compare interpretability and practicality using produced explanations, dependency and tuning burden, structural size and local fit/predict time; and
6. judge whether any machine-learning gain is sufficiently large and stable to justify added complexity.

The models are judged by accuracy, stability, ease of explanation and practical cost. The method must also be reproducible. Machine learning can represent nonlinear relationships and sequences, but it also adds overfitting risk and more choices. A complex method should therefore show a clear accuracy gain that remains in later checks.

Lagged linear regression is auxiliary rather than a fifth answer to the research question: it tests whether engineered features help without a nonlinear learner. Rolling is the persistence benchmark; GARCH is the conditional-variance benchmark; Random Forest is a nonlinear tabular learner; and LSTM is a nonlinear sequence learner. Their results compare complete forecasting pipelines, not isolated architectures. The asset is specifically Hyperliquid BTC perpetual futures, not a generic Bitcoin spot index, limiting generalisation to other venues and spot markets.

The evaluation date is fixed rather than recalculated as a moving 80/20 boundary after each download. 16 November 2025 remains the first forecast-origin date in the primary test whenever new completed candles are appended. This matters because otherwise a refresh would move several observations from test into training, changing both the evidence and the question. Applying the same final code to archives ending 19 July and 19 August 2026 tests whether 31 newly completed days change the result under the same historical cutoff.

2. Literature review

2.1 Why GARCH remains a serious benchmark

Financial return variance is not usually constant through time. Large movements often cluster together, as do calm periods. Bollerslev (1986) developed Generalised Autoregressive Conditional Heteroskedasticity so that conditional variance could depend on recent squared shocks and earlier conditional variance. GARCH(1,1) is therefore designed around persistence rather than being a generic regression applied to a time series.

Hansen and Lunde (2005) demonstrate why GARCH(1,1) should not be treated as a weak benchmark. Their peer-reviewed study compares 330 ARCH-type models out of sample and controls for data snooping, giving a credible broad test of forecast performance. However, its exchange-rate and IBM equity data do not transfer directly to Bitcoin perpetual futures: GARCH is hard to beat for the exchange rate but inferior for IBM. Katsiampa (2017) is more asset-relevant because it studies Bitcoin, but it compares only GARCH-family specifications and reports in-sample evidence. These complementary strengths and limits justify using GARCH as a serious benchmark, not assuming that it must win here.

2.2 Mixed evidence on machine learning

Dudek et al. (2024) provide a peer-reviewed comparison of 12 statistical and machine-learning methods across four cryptocurrencies, making the study directly relevant to whether greater complexity consistently improves volatility forecasts. Its cross-model, multi-asset design is stronger for this research question than a single-model success. However, its realised variance is constructed from intraday returns and is richer than the overlapping daily-close proxy used here; differences in target, horizon and sample prevent a direct replication. It is retained because the mixed rankings support testing an ML advantage rather than assuming one.

Huang, Sangiorgi and Urquhart (2024) provide peer-reviewed Bitcoin evidence that neural networks can outperform GARCH across their tested horizons. The Bitcoin focus is relevant, but their 2014-2021 high-frequency sample, richer transformations and many more intraday observations mean this small daily-data LSTM is not an equivalent replication. Shen, Wan and Leatham (2021) likewise report stronger average recurrent-network forecasts but weaker performance for extreme events and Value at Risk. That limitation makes the study useful here because it supports a separate high-volatility check rather than reliance on aggregate RMSE alone.

Zahid, Iqbal and Koutmos (2022) combine GARCH with machine learning, showing that the two are not opposing camps. Catania, Grassi and Ravazzolo (2019) also find model and parameter instability in cryptocurrency forecasting. This supports multiple chronological checks while recognising that one market history is not an independent replication.

2.3 Features, sequences and interpretability

Breiman (2001) describes Random Forest as an ensemble of randomised decision trees. Bootstrap sampling and random feature selection reduce dependence among trees, while averaging reduces variance. The method can capture thresholds and interactions without assuming a linear equation. In time-series work, however, it does not know chronological order automatically; past information must be represented through lags and rolling features. Feature design is therefore part of the model, not neutral preprocessing.

Hochreiter and Schmidhuber (1997) introduced LSTM to address the difficulty recurrent neural networks face when learning longer dependencies. Its gates control what information is retained, updated and exposed. That architecture appears suitable for persistent volatility, but suitability is not proof of accuracy. Sequence length, hidden size, scaling, learning rate, regularisation and stopping all matter, especially with only about one thousand training observations.

Interpretability must also be defined carefully. Lundberg and Lee (2017) show how attribution methods can explain parts of complex predictions, while Molnar (2025) stresses that different tools answer different questions. This project credits only evidence actually produced: equations and parameters for GARCH; signed standardised coefficients for linear regression; impurity and permutation importance plus out-of-bag (OOB) diagnostics for Random Forest; and architecture, losses, seed stability and post-hoc input-ablation sensitivity for LSTM. Importance and ablation are associational, can be distorted by correlated inputs and do not explain one forecast causally.

2.4 Volatility is measured through a proxy

True conditional volatility is latent. Patton (2011) is a peer-reviewed methodological study showing that model rankings can depend on the imperfect proxy and loss function used to compare forecasts. Its formal analysis directly supports reporting QLIKE on squared proxy and forecast values. However, it does not make this overlapping daily rolling standard deviation equivalent to latent variance, and intraday realised variance is usually richer than one daily close. The source is retained to discipline the comparison, not to validate the proxy; the target remains a *daily volatility proxy*.

The proxy has an important structural property: 29 of the 30 returns in tomorrow's primary window are already known today. Persistence is therefore partly mathematical as well as empirical. Rolling volatility is a demanding operational benchmark, and a model must estimate the effect of the entering return and the leaving return more accurately to beat it. This design is appropriate for an updated one-day-ahead risk indicator, but it is different from predicting a wholly future, non-overlapping 30-day window.

Overall, GARCH has a strong basis, machine learning can outperform it with richer data, simple methods can remain competitive, and rankings depend on the market, target and evaluation procedure.

Three expectations follow. Rolling volatility should be difficult to beat because of target overlap. GARCH may benefit if recent shocks help estimate the one unknown entering return. Random Forest and LSTM should help only if their nonlinear or sequential patterns add information beyond persistence. These are expectations to test, not assumptions about which model should win.

3. Mathematical formulation

Let P_t be the daily closing price on day t . The logarithmic return is

$$r_t = \ln\left(\frac{P_t}{P_{t-1}}\right).$$

For a window of n days, the daily realised-volatility proxy is the sample standard deviation

$$\text{RV}_t^{(n)} = \sqrt{\frac{1}{n-1} \sum_{i=t-n+1}^t (r_i - \bar{r}_t)^2}.$$

The primary window is $n = 30$, while $n = 14$ is a robustness check. Values are not annualised. A row formed at day t predicts

$$y_t = \text{RV}_{t+1}^{(n)},$$

and rolling historical volatility sets $\hat{y}_t = \text{RV}_t^{(n)}$.

Feature scaling is fitted only on the relevant training portion:

$$z_{tj} = \frac{x_{tj} - \mu_{j, \text{train}}}{\sigma_{j, \text{train}}}.$$

The lagged linear model then solves a lightly regularised objective,

$$\hat{\beta} = \operatorname{argmin}_{\beta} \sum_{t \in \text{train}} (y_t - z_t^\top \beta)^2 + \lambda \|\beta\|_2^2,$$

where $\lambda = 10^{-8}$ is used for numerical stability rather than strong shrinkage. Random Forest averages B fitted trees,

$$\hat{y}_t^{\text{RF}} = \frac{1}{B} \sum_{b=1}^B T_b(z_t).$$

For LSTM, input and previous hidden state determine gates such as $f_t = \sigma(W_f[x_t, h_{t-1}] + b_f)$; the cell is updated by $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$, and the final hidden representation is mapped to one volatility estimate. These equations describe information flow, not a direct economic interpretation of each learned weight.

GARCH(1,1) models one-step conditional variance as

$$h_{t+1} = \omega + \alpha \varepsilon_t^2 + \beta h_t.$$

Here α measures shock response and β persistence. The final estimates are $\alpha = 0.10$, $\beta = 0.78$ and $\alpha + \beta = 0.88$. The grid search selects α and β ; variance targeting implies $\omega = (1 - \alpha - \beta)\sigma_r^2$, so the three reported parameters are not three independently searched quantities.

To map conditional variance to the rolling target, let S_t and Q_t be the sum and sum of squares of the known preceding $n - 1$ returns. Under zero conditional mean,

$$E\left[s_{t+1}^2 \mid F_t\right] = \frac{Q_t + h_{t+1} - (S_t^2 + h_{t+1}) / n}{n - 1}.$$

Because the target is a standard deviation and squared-error evaluation is used, the primary implementation estimates $E[s \mid F_t]$ with deterministic 80-point Gauss-Hermite quadrature. The analytic $\sqrt{E[s^2 \mid F_t]}$ is retained as a sensitivity because the two expressions are not identical when the square root is concave.

For m test observations,

$$\text{MAE} = \frac{1}{m} \sum_{t=1}^m |y_t - \hat{y}_t|, \quad \text{RMSE} = \sqrt{\frac{1}{m} \sum_{t=1}^m (y_t - \hat{y}_t)^2}.$$

RMSE gives more influence to large misses; MAE represents a typical absolute miss. For QLIKE, the standard-deviation proxy is evaluated on the variance scale:

$$v_t = \max(y_t^2, \varepsilon), \quad \hat{v}_t = \max(\hat{y}_t^2, \varepsilon), \quad \text{QLIKE} = \frac{1}{m} \sum_{t=1}^m \left(\frac{v_t}{\hat{v}_t} - \ln \frac{v_t}{\hat{v}_t} - 1 \right),$$

where $\varepsilon = 10^{-12}$ prevents undefined values and lower is better. QLIKE is a relevant robustness metric for an imperfect proxy (Patton, 2011), but the paper's assumptions do not make this overlapping rolling proxy equivalent to latent conditional variance. Bootstrap replicate b compares each model with rolling using

$$\Delta_b = \text{RMSE}_b(M) - \text{RMSE}_b(\text{Rolling}),$$

so negative values favour the competing model.

4. Methodology

4.1 Data collection, quality and refresh control

Hyperliquid's public `candleSnapshot` endpoint supplied daily OHLCV candles (Hyperliquid, 2026). Of 1,272 returned rows, the one still-open candle was rejected using its end timestamp. The accepted archive contains 1,271 completed candles from 26 February 2023 through 19 August 2026 (2026-08-19).

The quality gate checks schema, chronology, cadence, symbol, interval, prices, OHLC ordering and activity; no critical issue was found. An open candle would change the newest return, proxy and activity fields simultaneously, so completion is enforced. Perpetual futures remain distinct from spot BTC-USD.

The 16 November 2025 cutoff was derived from the original chronological split and then frozen. New candles extend the test end rather than moving its start or becoming training rows. For a same-code comparison, the final pipeline was run on a slice ending 19 July and the current input ending 19 August. The 31 appended days provide a prospective-style stability check within the same market history.

4.2 Feature construction and leakage controls

Returns, activity variables, lags and rolling measures use only information observed by origin t ; the target is shifted to $t + 1$ afterwards. Exports retain both origin `date` and predicted `target_date`. Missing warm-up rows are removed before the chronological split.

`realised_volatility_30d` and `rolling_return_std_30d` were the same mathematical feature up to floating-point precision. Retaining both would duplicate persistence, split importance and increase selection probability inside a tree. The matching rolling-standard-deviation column is therefore excluded, leaving 25 tabular inputs and eight LSTM inputs.

The final 30-day frame contains 1,226 rows from 11 April 2023 to 18 August 2026. The fixed training portion contains 950 forecast-origin rows ending 15 November 2025; the test contains 276 origins from 16 November 2025 to 18 August 2026, with target dates through 19 August. No random train-test shuffle is used. Tabular scaling is fitted on training only. LSTM scaling is fitted before its chronological internal-validation block, so validation and test distributions cannot influence the scale parameters.

Effective training counts differ by model even though the cutoff is shared. Linear regression and Random Forest use 950 labelled rows. LSTM uses 921 chronological sequences, divided into 783 fitting sequences and 138 internal-validation sequences. GARCH estimates its return process from 993 pre-cutoff returns, including the warm-up history that cannot form a complete feature row. Rolling has no fitted sample. Reporting these counts avoids implying that structurally different models consume identical observations.

All models use the same cutoff, and no forecast uses later information. Their inputs are not identical. Linear regression and Random Forest use 25 prepared features, LSTM uses eight features in sequences, GARCH uses returns and rolling uses the current proxy. GARCH first forecasts conditional variance and converts it to the rolling target, while the supervised models predict that target directly. The results therefore compare the complete methods as implemented, not model type alone.

4.3 Models

Rolling historical volatility. The benchmark carries today's rolling proxy forward by one day. It has no fitted parameters and directly exploits the overlap between adjacent windows.

GARCH(1,1). A deterministic coarse-to-fine Gaussian maximum-likelihood grid search is combined with variance targeting. The likelihood scores r_t using h_t before r_t^2 updates h_{t+1} , preventing the current shock from entering its own variance. Parameters remain fixed throughout the primary test, but conditional variance updates when a new return becomes observable. Forecasts are aligned by date, and missing or duplicate mappings cause failure.

Lagged linear regression. A linear equation is fitted to standardised inputs with ridge 10^{-8} . Every coefficient is exported. Coefficients describe association on the standardised scale; correlated lagged measures still prevent causal interpretation.

Random Forest. The experimental model is scikit-learn 1.9.0's `sklearn.ensemble.RandomForestRegressor` (Pedregosa et al., 2011), as recorded in the saved run metadata, not a project-local approximation. Six predeclared combinations of maximum depth (5 , 8 , unrestricted) and minimum leaf size (5 , 10) use 300 trees and square-root feature sampling. The last 15% of pre-test rows is a chronological validation block; validation RMSE selects the candidate, with QLIKE only as a tie-break. The selected unrestricted-depth, leaf-size-5 configuration is refitted on all 950 pre-test rows with seed 42. Impurity importance, ten-repeat post-hoc holdout permutation importance and OOB diagnostics are exported.

LSTM. The network is implemented in PyTorch 2.13.0 as `torch.nn.LSTM` followed by a 16-unit ReLU head and one output (Paszke et al., 2019); the dependency version is recorded in the run metadata and training summary. Thirty-observation sequences feed this network. Four predeclared hidden-size/learning-rate pairs (16 or 32 ; 0.001 or 0.003) are compared on the last 15% of training sequences. Chronological validation MSE selects the candidate and controls early stopping; the final test is untouched. The selected model has 32 hidden units, learning rate 0.003 , batch size 32, weight decay 1e-5 , 5,921 trainable parameters and selected epoch 20. Seeds 7, 42 and 101 test optimisation sensitivity. Post-hoc feature ablation replaces one standardised input with its training mean at every sequence step; it is explanatory sensitivity, not another selection stage.

4.4 Evaluation and decision rules

The primary result is a fixed-cutoff chronological holdout. Model parameters and weights are not refitted daily, although every method receives information available at the forecast origin. RF and LSTM candidate sets are deliberately compact and comparable in principle rather than equal in size; both use the same training-only chronological-validation rule and never inspect the final holdout. GARCH uses a deterministic likelihood grid, rolling has no selection and linear regression has only a fixed numerical ridge. A supplementary expanding-window design divides the test into four ordered blocks and refits each fitted pipeline at later boundaries.

Robustness also includes 14- and 30-day targets, two chronological test halves, and low-, medium- and high-volatility terciles defined from realised test targets. The regimes are ex-post diagnostic groups, not real-time classifications. A paired circular moving-block bootstrap resamples blocks of 30 days 2,000 times. Thirty days matches the main target's overlap length and helps retain local dependence (Künsch, 1989). Percentile intervals describe uncertainty under this resampling design; they are not universal significance guarantees.

The four comparison areas are reported separately:

Dimension	Evidence used	Decision rule
Accuracy	RMSE, MAE and variance-scale QLIKE	RMSE sets the displayed rank; MAE or QLIKE disagreements must be discussed
Robustness	Target windows, halves, folds, regimes, refresh and bootstrap	A claimed advantage should not depend on one boundary or a small set of dates
Interpretability	Formula, parameters, coefficients, importance or recorded architecture	Only explanations actually produced by the pipeline are credited
Practicality	Fit/predict time, dependencies and structural size	Timings are local implementation evidence, not universal speed claims
Reproducibility	Fixed dates, seeds, configuration, metadata and tests	Another run should recover the design and explain any data-driven change

A complex model is justified only if its gain is sufficiently large and stable to compensate for weaker transparency or greater implementation burden.

5. Results

5.1 Primary 30-day target

Rank	Model	MAE	RMSE	QLIKE	RMSE vs rolling
1	GARCH(1,1)	0.00048109	0.00098301	0.00445845	-29.569%
2	Lagged linear regression	0.00072090	0.00137492	0.00724584	-1.489%
3	Rolling historical volatility	0.00061444	0.00139570	0.00754794	0.000%
4	LSTM	0.00099818	0.00168735	0.00897661	+20.896%
5	Random Forest	0.00123057	0.00211129	0.01182354	+51.271%

GARCH has the lowest MAE, RMSE and QLIKE. QLIKE gives the same complete ordering as RMSE, so the result is not an artefact of using only standard-deviation-scale squared error. Its -0.00041269 RMSE difference is a 29.569% reduction from rolling.

Linear ranks second by RMSE and QLIKE but has higher MAE than rolling, so its small gain needs uncertainty evidence. LSTM and Random Forest are +0.00029165 and +0.00071559 above rolling by RMSE.

After de-duplication, both forest importance methods rank current 30-day volatility, its one-day lag, 30-day mean absolute return and its two-day volatility lag highest, in that order. These are associations, not causes; the forest largely reconstructs persistence more compactly expressed by simpler models.

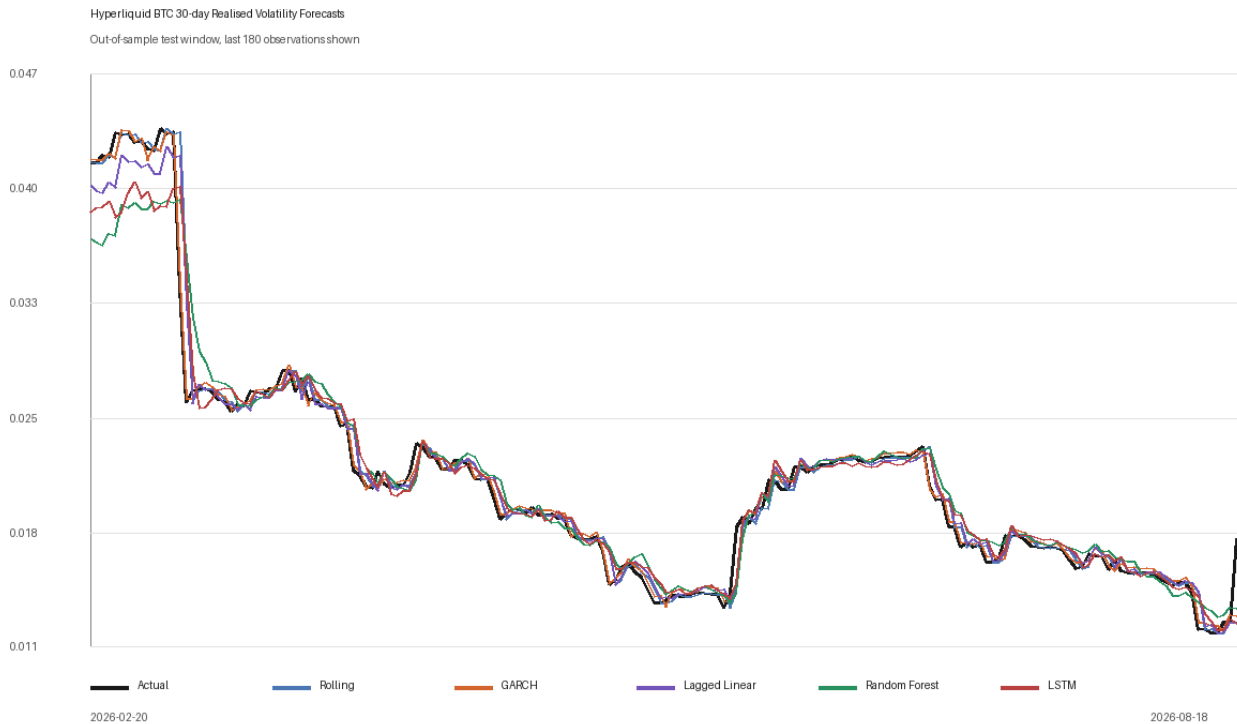


Figure 1. Actual next-day rolling-volatility proxies and forecasts over the displayed test interval. The underlying CSV retains every test observation; the chart limits the visible period for legibility.

Forecasts overlap during persistent stretches and separate most at level changes. Visual inspection is therefore insufficient: RMSE emphasises sharp misses, while MAE, regime bias and resampled differences reveal evidence hidden by the compressed plot.

5.2 Target, time and uncertainty robustness

Target window	GARCH RMSE	Linear RMSE	Rolling RMSE	LSTM RMSE	Random Forest RMSE
14 days	0.00181314	0.00258160	0.00272309	0.00352137	0.00383276
30 days	0.00098301	0.00137492	0.00139570	0.00168735	0.00211129

GARCH ranks first at 14 days, 33.416% below rolling. This shorter window has larger errors for every model because each entering or leaving return has greater influence. GARCH also ranks first in both test halves, with RMSE 0.00121469 and 0.00067613 .

Model	Point RMSE difference vs rolling	30-day block-bootstrap interval	Resamples favouring model
GARCH(1,1)	-0.00041269	[-0.00089954, -0.00016861]	100.00%
Lagged linear regression	-0.00002078	[-0.00008328, 0.00004204]	74.80%
LSTM	+0.00029165	[0.00000368, 0.00061163]	2.30%
Random Forest	+0.00071559	[0.00005958, 0.00128485]	0.15%

GARCH's interval remains below zero; linear's crosses zero. Both machine-learning intervals remain above zero: only 2.30% of resamples favour LSTM and 0.15% favour Random Forest. This supports aggregate underperformance rather than a few isolated misses, although resampling one history does not create independent markets.

5.3 Rolling origin, regimes and diagnostics

Concatenated fold RMSE is 0.00098432 , 0.00137390 , 0.00139570 , 0.00189862 and 0.00209190 for GARCH, linear, rolling, LSTM and Random Forest. Their QLIKE values are 0.00446344 , 0.00721327 , 0.00754794 , 0.00998982 and 0.01161326 , giving the same ordering.

LSTM is locally competitive only in the first fold, where its RMSE is 0.00082244 and it ranks second, just ahead of linear, Random Forest and rolling. It ranks fourth in folds 2 and 3 and fifth in fold 4, so this one local gain does not establish aggregate stability.

GARCH's low-, medium- and high-regime RMSE values are 0.00069948 , 0.00068656 and 0.00139222 . Its -0.00013275 high-regime bias indicates slight average underprediction when risk matters most.

Random Forest OOB RMSE is 0.00128268 across all 950 training rows, versus 0.00211129 chronologically, a 0.00082861 gap. OOB trees can train on later training-period rows, so this diagnoses internal prediction rather than time validation and exposes weaker cross-period generalisation.

Seeds 7, 42 and 101 yield LSTM RMSE 0.00178061 , 0.00168735 and 0.00172688 , selecting epochs 11, 20 and 33. The 0.00009326 range shows optimisation variation, but every run remains above rolling's 0.00139570 .

5.4 One-month fixed-cutoff refresh

Model	RMSE through 19 July	RMSE through 19 August	Rank change
GARCH(1,1)	0.00098502	0.00098301	1 → 1
Lagged linear regression	0.00140087	0.00137492	2 → 2
Rolling historical volatility	0.00142744	0.00139570	3 → 3
LSTM	0.00174351	0.00168735	4 → 4
Random Forest	0.00220594	0.00211129	5 → 5

The added 31 days lower overall RMSE by 0.204% to 4.291% and leave every rank unchanged. QLIKE rises for every model, however, so not every loss measure improves. This remains a procedural stability check within one market history, not an independent replication.

The last target needs to be read separately. BTC closed at 69,323 on 19 August, up from 64,696 on 18 August. The log return was 0.06908 , and the 30-day proxy jumped from 0.0121249 to 0.0174694 . Its forecast origin was 18 August, before that return was observable; all five forecasts were between about 0.0120 and 0.0130 . All five models therefore missed the jump. This one day does not change the full-period ranking, and it would be wrong to tune a revised model on this holdout row and then call the same row independent test evidence.

5.5 GARCH conversion sensitivity

Conversion from conditional variance	MAE	RMSE	QLIKE
$E[s F_t]$, 80-point Gauss-Hermite	0.00048109	0.00098301	0.00445845
$\sqrt{E[s^2 F_t]}$, analytic sensitivity	0.00048653	0.00098255	0.00445288

The analytic sensitivity has mean forecasts 0.00000979 higher, RMSE 0.00000046 lower and QLIKE 0.00000557 lower. The difference is very small, and GARCH remains first under both conversions. Quadrature remains primary because it estimates the conditional mean of the standard-deviation target.

5.6 Practicality and interpretability

Model	Local fit / predict	Selection and dependency	Structural and explanation evidence
Rolling	0.000000 / 0.000011 s	None; NumPy/Pandas	0 parameters; direct persistence rule
Linear regression	0.004307 / 0.000019 s	Fixed numerical ridge; NumPy	26 coefficients; every signed coefficient exported
GARCH(1,1)	0.529684 / 0.010776 s	Deterministic likelihood grid; NumPy	3 reported parameters; shock response and persistence
Random Forest	1.384384 / 0.007390 s	6 validation candidates; scikit-learn	57,970 nodes; importance and OOB evidence
LSTM	9.739972 / 0.004044 s	4 candidates plus early stopping; PyTorch	5,921 weights; loss, seed and ablation evidence

Runtime alone does not establish deployability. Rolling and linear require almost no tuning; GARCH adds a deterministic grid and target conversion; Random Forest adds a library dependency and six candidates; LSTM adds PyTorch, gradient optimisation, early stopping and seed sensitivity. These are local implementation measurements, not universal claims that GARCH is always computationally faster.

The largest absolute non-intercept linear coefficients on standardised inputs are:

Feature	Coefficient
Current 30-day proxy	+0.00603107
14-day return standard deviation	+0.00058092
30-day mean absolute return	+0.00039447
7-day mean absolute return	+0.00037525
One-day proxy lag	-0.00032560
7-day return standard deviation	-0.00030675

Correlated rolling features make individual signs conditional rather than causal. Forest impurity and permutation rankings both place the current proxy first. For LSTM, replacing the current standardised proxy with its training mean raises RMSE by 0.00476927; ablating 30-day mean absolute return raises it by 0.00083285. These ablation results quantify fitted-prediction sensitivity to the inputs; they do not explain the recurrent state or establish causality.

6. Discussion

GARCH performs best on MAE, RMSE and QLIKE and stays first across both target windows, both test halves, every expanding-window block and every volatility group. Rolling is the simplest benchmark. Linear regression is fast and easy to inspect, but its 1.489% RMSE improvement comes with worse MAE and a bootstrap interval that crosses zero. Random Forest and LSTM have higher overall error and require more model choices.

The comparison does not combine interpretability and runtime into a single score. Error improvement is considered first, followed by whether it persists in the other checks and the extra explanation and implementation burden required.

GARCH's performance is theoretically plausible. Conditional variance responds to the latest squared shock while retaining earlier variance through β . Its persistence of 0.88 indicates substantial memory while remaining below the stationary boundary. The rolling-target conversion also gives GARCH a relevant task: 29 returns in the next 30-day window are known, while uncertainty about the entering return is supplied by h_{t+1} . Rolling assumes that the incoming observation produces no predictable update; GARCH replaces that assumption with an estimated variance contribution.

The Gauss-Hermite check qualifies this explanation. The primary expected-standard-deviation forecast is not algebraically identical to the square root of expected variance. Their mean forecasts differ by only 0.00000979, their RMSE values differ by 0.00000046, and GARCH remains first under either conversion. This small sensitivity leaves the primary conclusion unchanged while preserving the distinction between the two conversions.

The rolling benchmark's strength is partly built into the target. Tomorrow's window overlaps heavily with today's, so persistence is a mathematical consequence of target construction as well as a market pattern. This makes rolling fair for a one-day operational update, but it may understate the value of richer models for a wholly future, non-overlapping horizon. It also explains why a very small RMSE advantage should not automatically be called practically important.

Linear regression illustrates the difference between ranking and meaningful improvement. Its RMSE is 1.489% below rolling, but its MAE is higher, and its bootstrap interval crosses zero. Calling it definitively better would therefore overstate the evidence. Removing the exactly duplicated rolling-standard-deviation feature makes coefficient evidence less misleading, but remaining lag and window variables are still correlated. Signs are auditable associations, not independent causal effects.

Random Forest does not beat the simpler models overall. Its most important features are mainly measures of recent volatility, so much of the forest is still learning persistence. The gap between out-of-bag and later-period error also suggests weaker transfer across market periods. Feature importance shows association, not the cause of one individual forecast.

LSTM provides a more nuanced case. It ranks second in the first expanding-window fold, narrowly ahead of linear, Random Forest and rolling, but drops to fourth or fifth in the other folds. Its aggregate RMSE and QLIKE, high-volatility behaviour and seed range show that the local advantage is not dependable enough here. Ablation shows strong dependence on the current proxy, not a transparent mechanism for each forecast. The conclusion is limited to the current daily-data, direct-target LSTM pipeline, which supplies no robust benefit in these tests.

With the cutoff frozen, the 31-day refresh leaves every rank unchanged; RMSE falls slightly for every model while QLIKE rises.

7. Conclusion

This project asked to what extent Random Forest and LSTM justify their additional complexity over rolling historical volatility and GARCH(1,1) when forecasting the next-day update of a Hyperliquid BTC perpetual-futures volatility proxy. Under the tested design, they do not. GARCH has the lowest MAE, RMSE (0.00098301) and QLIKE (0.00445845), and its RMSE is 29.569% below rolling. LSTM and Random Forest both remain worse than rolling overall; linear regression's small RMSE gain comes with worse MAE and a bootstrap interval that crosses zero.

The ranking is robust within this dataset. GARCH remains first for both target windows, both test halves, all four expanding-window folds and all three volatility regimes, and its block-bootstrap RMSE-difference interval remains below zero. In contrast, LSTM is competitive in only one fold, both machine-learning intervals remain above zero, and every LSTM seed remains worse than rolling. These checks strengthen the answer, but they still resample or partition one market history rather than provide independent-market replication.

Interpretability and practicality reinforce the accuracy result. GARCH has three reported parameters and a short explanation through shock response and persistence; its deterministic grid takes about 0.53 seconds to fit locally. Random Forest adds six validation candidates, scikit-learn and 57,970 tree nodes, while its importance measures remain global and associational. LSTM adds PyTorch, four candidates, early stopping, seed sensitivity and 5,921 weights, while ablation does not reveal a causal recurrent mechanism. Rolling remains the simplest fallback. Because neither complex model delivers a stable accuracy gain to compensate for these explanation and implementation burdens, their additional complexity is not justified here.

The conclusion is limited to one asset, one exchange, an overlapping daily proxy, one market history, a Gaussian GARCH likelihood, compact machine-learning searches and pipelines with different inputs and transformations. A stronger extension would reserve another untouched future period, construct realised variance from intraday returns, test Student-t or asymmetric GARCH, and tune machine learning within nested chronological validation. A hybrid using GARCH variance as a nonlinear-model input could also be tested, but each extension should be introduced separately so its effect remains identifiable.

Reviewing this work changed my standard for accepting a result. Reproducibility became part of the evidence rather than an appendix to it: date mapping, the GARCH update order, target conversion, open-candle filtering and a duplicated feature all had to be checked before the ranking was trustworthy. Replacing the project-local forest with scikit-learn, freezing the cutoff and adding QLIKE also showed that a complex model or one favourable fold is not enough; a conclusion should follow transparent held-out evidence and state what the design cannot prove.

References

- Bollerslev, T. (1986) 'Generalized autoregressive conditional heteroskedasticity', *Journal of Econometrics*, 31(3), pp. 307–327. Available at: [https://doi.org/10.1016/0304-4076\(86\)90063-1](https://doi.org/10.1016/0304-4076(86)90063-1).
- Brauneis, A. and Sahiner, M. (2026) 'Crypto volatility forecasting: Mounting a HAR, sentiment, and machine learning horserace', *Asia-Pacific Financial Markets*, 33, pp. 379–411. Available at: <https://doi.org/10.1007/s10690-024-09510-6>.
- Breiman, L. (2001) 'Random forests', *Machine Learning*, 45, pp. 5–32. Available at: <https://doi.org/10.1023/A:1010933404324>.
- Catania, L., Grassi, S. and Ravazzolo, F. (2019) 'Forecasting cryptocurrencies under model and parameter instability', *International Journal of Forecasting*, 35(2), pp. 485–501. Available at: <https://doi.org/10.1016/j.ijforecast.2018.09.005>.

- Dudek, G., Fiszeder, P., Kobus, P. and Orzeszko, W. (2024) 'Forecasting cryptocurrencies volatility using statistical and machine learning methods: A comparative study', *Applied Soft Computing*, 151, 111132. Available at: <https://doi.org/10.1016/j.asoc.2023.111132>.
- Hansen, P.R. and Lunde, A. (2005) 'A forecast comparison of volatility models: Does anything beat a GARCH(1,1)?', *Journal of Applied Econometrics*, 20(7), pp. 873–889. Available at: <https://doi.org/10.1002/jae.800>.
- Hochreiter, S. and Schmidhuber, J. (1997) 'Long short-term memory', *Neural Computation*, 9(8), pp. 1735–1780. Available at: <https://doi.org/10.1162/neco.1997.9.8.1735>.
- Huang, Z.-C., Sangiorgi, I. and Urquhart, A. (2024) 'Forecasting Bitcoin volatility using machine learning techniques', *Journal of International Financial Markets, Institutions and Money*, 97, 102064. Available at: <https://doi.org/10.1016/j.intfin.2024.102064>.
- Hyperliquid (2026) 'Info endpoint', *Hyperliquid Docs*. Available at: <https://hyperliquid.gitbook.io/hyperliquid-docs/for-developers/api/info-endpoint> (Accessed: 20 August 2026).
- Katsiampa, P. (2017) 'Volatility estimation for Bitcoin: A comparison of GARCH models', *Economics Letters*, 158, pp. 3–6. Available at: <https://doi.org/10.1016/j.econlet.2017.06.023>.
- Künsch, H.R. (1989) 'The jackknife and the bootstrap for general stationary observations', *The Annals of Statistics*, 17(3), pp. 1217–1241. Available at: <https://doi.org/10.1214/aos/1176347265>.
- Lundberg, S.M. and Lee, S.-I. (2017) 'A unified approach to interpreting model predictions', *Advances in Neural Information Processing Systems*, 30. Available at: <https://arxiv.org/abs/1705.07874>.
- Molnar, C. (2025) *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. 3rd edn. Available at: <https://christophm.github.io/interpretable-ml-book/> (Accessed: 20 July 2026).
- Paszke, A. et al. (2019) 'PyTorch: An imperative style, high-performance deep learning library', *Advances in Neural Information Processing Systems*, 32. Available at: https://papers.neurips.cc/paper_files/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html.
- Patton, A.J. (2011) 'Volatility forecast comparison using imperfect volatility proxies', *Journal of Econometrics*, 160(1), pp. 246–256. Available at: <https://doi.org/10.1016/j.jeconom.2010.03.034>.
- Pedregosa, F. et al. (2011) 'Scikit-learn: Machine Learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830. Available at: <https://www.jmlr.org/papers/v12/pedregosa11a.html>.
- Shen, Z., Wan, Q. and Leatham, D.J. (2021) 'Bitcoin return volatility forecasting: A comparative study between GARCH and RNN', *Journal of Risk and Financial Management*, 14(7), 337. Available at: <https://doi.org/10.3390/jrfm14070337>.
- Zahid, M., Iqbal, F. and Koutmos, D. (2022) 'Forecasting Bitcoin volatility using hybrid GARCH models with machine learning', *Risks*, 10(12), 237. Available at: <https://doi.org/10.3390/risks10120237>.